Lecture 12: Support Vector Machines — Rigorous Pedagogical Explanation

1 High-Level Overview

Support Vector Machines (SVMs) are supervised learning models for binary classification. The central idea is geometrically elegant: rather than fitting a decision boundary that merely separates classes, SVM finds the unique hyperplane that maximizes the margin — the gap between the two classes. This margin maximization is what gives SVM its strong generalization guarantees.

Where SVM fits in the broader ML landscape:

SVMs belong to the family of discriminative classifiers that learn decision boundaries directly (as opposed to generative models like LDA/QDA that model class-conditional distributions). SVMs are notable because:

- Their solution is governed by only a small subset of training points (the support vectors)
- The learning problem is a convex quadratic program with a unique global optimum
- Through the kernel trick, they extend naturally to nonlinear boundaries without explicitly computing high-dimensional feature maps

2 Slide-by-Slide Explanation

Slide 5: SVM vs. Other Classifiers

This slide contrasts SVM with classifiers already covered in the course.

Method	**Decision Rule**	**Parameters Learned From**
KNN	Majority vote among k nearest neighbors	No explicit training; uses all data at inference
Logistic Regression	Sigmoid of linear combination	MLE of $p(y_c \mid x_c, \beta)$
LDA/QDA	Linear/quadratic boundary	Class means and covariances
SVM	Sign of $\mathbf{w}^T\mathbf{x} + b$	Hyperplane that maximally separates classes

Key distinction: SVM explicitly models the interaction between classes through the geometry of the separating hyperplane. The parameters $\mathbf{w}$ and b are chosen so that the boundary is as far as possible from both classes simultaneously.

Slide 8: The Idea of SVM

Classification function:

$$f(\mathbf{x}_j) = \hat{\mathbf{w}}^T\mathbf{x}_j + b$$

Variable definitions:

- $\mathbf{x}_j \in \mathbb{R}^n$: the n-dimensional input feature vector for the j-th sample

- $\hat{\mathbf{w}} \in \mathbb{R}^n$: the weight vector (normal to the decision hyperplane)
- $b \in \mathbb{R}$: the bias (scalar offset that shifts the hyperplane from the origin)
- $f(\mathbf{x}_j)$: the decision value — its sign determines the predicted class

Geometric interpretation: The equation $\mathbf{w}^T\mathbf{x}+b = 0$ defines a hyperplane in $\mathbb{R}^n$. Points with $f(\mathbf{x}) > 0$ fall on one side; points with $f(\mathbf{x}) < 0$ fall on the other. The key question becomes: which hyperplane is optimal? Unlike logistic regression, which chooses any hyperplane that fits the likelihood, SVM selects the one that maximizes the distance to the nearest training points of each class.

Slide 9: Components of the SVM

The SVM constructs three parallel hyperplanes:

- Decision boundary: $\mathbf{w}^T\mathbf{x} + b = 0$
- Positive support hyperplane: $\mathbf{w}^T\mathbf{x}_+ + b = 1$
- Negative support hyperplane: $\mathbf{w}^T\mathbf{x}_- + b = -1$

Support vectors are the training points that lie exactly on the two bounding hyperplanes — they are the closest points to the decision boundary from each class. All other points are strictly further away and therefore do not directly influence the position of the boundary.

Why normalize to ± 1? This is a canonical rescaling. Given any separating hyperplane $\mathbf{w}^T\mathbf{x} + b = 0$, we can always rescale $\mathbf{w}$ and b so that the nearest points of each class satisfy the ± 1 conditions. This normalization is without loss of generality and simplifies the math.

Slide 10: Maximizing the Margin

Derivation of the margin: The perpendicular distance from a point $\mathbf{x}$ to the hyperplane $\mathbf{w}^T\mathbf{x} + b = 0$ is:

$$\text{distance} = \frac{|\mathbf{w}^T\mathbf{x} + b|}{\|\mathbf{w}\|_2}$$

The distance between the two support hyperplanes is:

$$d = \frac{|\mathbf{w}^T(\mathbf{x}_+ - \mathbf{x}_-)|}{\|\mathbf{w}\|_2} = \frac{|(1-b) - (-1-b)|}{\|\mathbf{w}\|_2} = \frac{2}{\|\mathbf{w}\|_2}$$

Step-by-step:

1. From the positive support plane: $\mathbf{w}^T\mathbf{x}_+ + b = 1 \Rightarrow \mathbf{w}^T\mathbf{x}_+ = 1 - b$
2. From the negative support plane: $\mathbf{w}^T\mathbf{x}_- + b = -1 \Rightarrow \mathbf{w}^T\mathbf{x}_- = -1 - b$
3. Difference: $\mathbf{w}^T(\mathbf{x}_+ - \mathbf{x}_-) = (1-b) - (-1-b) = 2$
4. Dividing by $\|\mathbf{w}\|_2$ gives the geometric margin $d = \frac{2}{\|\mathbf{w}\|_2}$

Optimization equivalence:

$$\text{Maximize } \frac{2}{\|\mathbf{w}\|_2} \equiv \text{Minimize } \|\mathbf{w}\|_2 \equiv \text{Minimize } \frac{1}{2}\|\mathbf{w}\|_2^2$$

The factor of $\frac{1}{2}$ is a standard convenience that simplifies gradient computations. Squaring the norm makes the objective differentiable and strictly convex.

Slide 11: Class Mapping Constraints

Given m training pairs $(\mathbf{x}_j, y_j)$ where $y_j \in \{-1, +1\}$:

- For positive class ($y_j = +1$): we require $f(\mathbf{x}_j) \geq +1$, i.e., $\mathbf{w}^T\mathbf{x}_j + b \geq 1$
- For negative class ($y_j = -1$): we require $f(\mathbf{x}_j) \leq -1$, i.e., $\mathbf{w}^T\mathbf{x}_j + b \leq -1$

Both conditions unify elegantly into a single constraint:

$$y_j(\mathbf{w}^T\mathbf{x}_j + b) \geq 1 \quad \forall j$$

Why this works: When $y_j = +1$ and $\mathbf{w}^T\mathbf{x}_j + b \geq 1$, the product is ≥ 1. When $y_j = -1$ and $\mathbf{w}^T\mathbf{x}_j + b \leq -1$, multiplying both sides by -1 gives $(-1)(\mathbf{w}^T\mathbf{x}_j + b) \geq 1$. So the single constraint captures both cases — a clean algebraic trick.

Slide 12–13: The Hard Margin SVM (Primal Problem)

The complete hard margin optimization problem:

$$\hat{b}, \hat{\mathbf{w}} = \arg\min_{b \in \mathbb{R}, \mathbf{w} \in \mathbb{R}^n} \frac{1}{2}\|\mathbf{w}\|_2^2 \quad \text{s.t.} \quad y_j(\mathbf{w}^T\mathbf{x}_j + b) \geq 1 \quad \forall j = 1, \ldots, m$$

Properties of this problem:

- Objective: Convex quadratic ($\frac{1}{2}\|\mathbf{w}\|_2^2$ is a quadratic form with positive definite Hessian I)
- Constraints: Linear in $\mathbf{w}$ and b
- Problem class: Convex Quadratic Program (QP) — guaranteed to have a unique global minimum
- Degrees of freedom: $n + 1$ (the n components of $\mathbf{w}$ plus b)

Training vs. Inference:

Phase	**Action**
Training	Solve the QP to obtain $\hat{\mathbf{w}}$ and $\hat{b}$
Inference	Compute $f(\mathbf{x}) = \hat{\mathbf{w}}^T\mathbf{x} + \hat{b}$; predict $\text{sign}(f(\mathbf{x}))$

Slides 14–19: The Hard Margin Problem and Its Failure

Core issue: The hard margin constraint $y_j(\mathbf{w}^T\mathbf{x}_j + b) \geq 1$ demands perfect linear separability. In practice:

- Data may not be linearly separable due to noise or class overlap — no feasible solution exists
- Even if separable, a single outlier can drastically tilt the hyperplane, leading to overfitting to that noise point

The slide sequence visually demonstrates this: adding one anomalous (noisy) training point forces the hard margin hyperplane to rotate dramatically, reducing the margin width and degrading generalization. This is the low robustness of the hard margin.

Intuition from the "red ball" visualization: The red ball represents the smallest perturbation of a test point that would cause misclassification. With a tilted hyperplane from noisy training, the margin shrinks and many test points fall within this perturbation radius — the classifier becomes fragile.

Slide 20: The Soft Margin SVM

To handle noise and non-separability, we introduce slack variables $\xi_j \geq 0$ (denoted z_j in the slides):

$$\hat{b}, \hat{\mathbf{w}} = \arg\min_{b,\mathbf{w},\mathbf{z}} \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{j=1}^{m} z_j$$

$$\text{s.t.} \quad y_j(\mathbf{w}^T\mathbf{x}_j + b) \geq 1 - z_j, \quad z_j \geq 0 \quad \forall j$$

Variable definitions:

- $z_j \geq 0$: slack variable for sample j — measures the degree of constraint violation
- $C > 0$: regularization parameter — controls the penalty for violations

Interpretation of z_j:

Value	Meaning
$z_j = 0$	Point correctly classified and on or beyond the margin
$0 < z_j \leq 1$	Point inside the margin but on the correct side of the boundary
$z_j > 1$	Point on the wrong side of the boundary (misclassified)

The objective has two competing terms:

- $\frac{1}{2}\|\mathbf{w}\|_2^2$: encourages a wide margin (low model complexity)
- $C\sum_j z_j$: penalizes margin violations (encourages correct classification)

Slide 21: Effect of the Regularization Parameter C

C value	Effect on z_j	Constraint enforced	Margin	Behavior
Small	Less penalized $\rightarrow$ large z_j	Relaxed	Wide	Underfitting
Large	More penalized $\rightarrow$ small z_j	Strictly enforced	Narrow	Overfitting

Conceptual link to regularization: This is precisely the bias-variance tradeoff instantiated in the SVM context:

- Small C: high bias, low variance (wide margin, ignores many points)
- Large C: low bias, high variance (tight fit to training data, sensitive to outliers)

As $C \rightarrow \infty$, the soft margin recovers the hard margin (zero violations tolerated).

Slides 24–27: The Dual Problem

Why Transform to the Dual? The primal SVM operates in weight space ($\mathbf{w} \in \mathbb{R}^n$). The dual reformulation operates in sample space ($\boldsymbol{\alpha} \in \mathbb{R}^m$). There are two key motivations:

Reason 1 — Efficiency: Many $\hat{\alpha}_j = 0$ at the optimum. This means $\hat{\mathbf{w}} = \sum_j \hat{\alpha}_j y_j \mathbf{x}_j$ depends only on a sparse subset of training points — the support vectors. This is computationally efficient when $m \ll n$ (few samples, high-dimensional features).

Reason 2 — The Kernel Trick: The dual problem depends on training data only through inner products $\mathbf{x}_i^T\mathbf{x}_j$. Replacing these with a kernel $K(\mathbf{x}_i, \mathbf{x}_j)$ implicitly maps data to a high-dimensional space without ever computing the mapping explicitly.

The Dual Problem Formulation:

$$\arg\max_{\boldsymbol{\alpha},\boldsymbol{\beta}\in\mathbb{R}^m} \mathbf{1}^T\boldsymbol{\alpha} - \frac{1}{2}\sum_{i=1}^m\sum_{j=1}^m \alpha_i\alpha_j y_i y_j \mathbf{x}_i^T\mathbf{x}_j$$

$$\text{s.t.} \quad \sum_{j=1}^m \alpha_j y_j = 0, \quad \alpha_j = C - \beta_j, \quad \alpha_j, \beta_j \geq 0$$

Variable definitions:

- $\alpha_j \geq 0$: Lagrange multiplier for the j-th margin constraint
- $\beta_j \geq 0$: Lagrange multiplier for the j-th non-negativity constraint on z_j

The constraint $\alpha_j = C - \beta_j$ with $\alpha_j, \beta_j \geq 0$ implies $0 \leq \alpha_j \leq C$.

Recovery of primal variables from dual:

$$\hat{\mathbf{w}} = \sum_{j=1}^m \hat{\alpha}_j y_j \mathbf{x}_j, \qquad \hat{b} = \frac{1}{m}\sum_j \left(y_j - \mathbf{x}_j^T\hat{\mathbf{w}}\right)$$

Slides 30–32: Derivation — Primal to Dual via Lagrangian

This is the core mathematical machinery. Let's trace through carefully.

Step 1: Form the Lagrangian Attach multipliers $\alpha_j \geq 0$ to the margin constraints and $\beta_j \geq 0$ to the non-negativity constraints:

$$L(b, \mathbf{w}, \mathbf{z}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_j z_j - \sum_j \alpha_j \left(y_j(\mathbf{x}_j^T\mathbf{w} + b) - 1 + z_j\right) - \sum_j \beta_j z_j$$

Step 2: Minimize over primal variables

- Over b: $\frac{\partial L}{\partial b} = -\sum_j \alpha_j y_j = 0 \Rightarrow \boxed{\sum_j \alpha_j y_j = 0}$
- Over z_j: $\frac{\partial L}{\partial z_j} = C - \alpha_j - \beta_j = 0 \Rightarrow \boxed{\alpha_j + \beta_j = C}$
- Over $\mathbf{w}$: $\frac{\partial L}{\partial w_i} = w_i - \sum_j \alpha_j y_j x_{ji} = 0 \Rightarrow \boxed{\mathbf{w} = \sum_j \alpha_j y_j \mathbf{x}_j}$

Step 3: Substitute back Substituting $\mathbf{w} = \sum_j \alpha_j y_j \mathbf{x}_j$ back into the Lagrangian:

$$\inf_{\mathbf{w}} L = \frac{1}{2}\mathbf{w}^T\mathbf{w} - \mathbf{w}^T\mathbf{w} + \sum_j \alpha_j = -\frac{1}{2}\|\mathbf{w}\|_2^2 + \sum_j \alpha_j$$

Expanding $\|\mathbf{w}\|_2^2 = \sum_i\sum_j \alpha_i\alpha_j y_i y_j \mathbf{x}_i^T\mathbf{x}_j$, we arrive at the dual objective.

Slides 33–35: KKT Conditions and Three Groups of Points

The Karush-Kuhn-Tucker (KKT) conditions are necessary (and for convex QPs, sufficient) optimality conditions. They consist of four groups: 1. Primal feasibility: Constraints must be satisfied at the solution 2. Dual feasibility: $\sum_j \alpha_j y_j = 0$, $0 \leq \alpha_j \leq C$ 3. Stationarity: Gradient of Lagrangian w.r.t. primal variables $= 0$ 4. Complementary slackness: $\alpha_j h_j(\hat{p}) = 0$ and $\beta_j z_j = 0$

The complementary slackness conditions partition all training points into exactly three groups:

Group 1: Correct side of margin ($\alpha_j = 0$)

$$\alpha_j = 0 \Rightarrow \beta_j = C \Rightarrow z_j = 0 \Rightarrow y_j(\mathbf{w}^T\mathbf{x}_j + b) \geq 1$$

These points are well inside their respective half-spaces, well beyond the margin. Since $\alpha_j = 0$, they contribute nothing to $\hat{\mathbf{w}}$ — they are irrelevant to the solution.

Group 2: On the margin boundary ($0 < \alpha_j < C$)

$$0 < \alpha_j < C \Rightarrow \alpha_j, \beta_j > 0 \Rightarrow z_j = 0 \Rightarrow y_j(\mathbf{w}^T\mathbf{x}_j + b) = 1$$

These are the support vectors — they lie exactly on the margin hyperplanes. They have nonzero α_j and directly define $\hat{\mathbf{w}}$.

Group 3: Wrong side of margin ($\alpha_j = C$)

$$\alpha_j = C \Rightarrow \beta_j = 0 \Rightarrow z_j > 0 \Rightarrow y_j(\mathbf{w}^T\mathbf{x}_j + b) = 1 - z_j$$

These points violate the margin (either inside the margin or misclassified). They also contribute to $\hat{\mathbf{w}}$ with $\alpha_j = C$.

Key insight: Only Groups 2 and 3 contribute to $\hat{\mathbf{w}} = \sum_j \hat{\alpha}_j y_j \mathbf{x}_j$. In practice, this is a small fraction of training data — this sparsity is what makes SVM efficient at inference.

Slides 37–40: Kernels and the Kernel Trick

The Problem with Linear SVM When classes are not linearly separable in the original feature space $\mathbb{R}^n$, we need a nonlinear decision boundary. One approach: explicitly map $\mathbf{x} \to \boldsymbol{\phi}(\mathbf{x}) \in \mathbb{R}^d$ where $d \gg n$, then apply a linear SVM in the high-dimensional space. **Problem:** If d is very large (or infinite), computing $\boldsymbol{\phi}(\mathbf{x})$ explicitly is intractable.

The Kernel Trick Observe that the dual problem only involves $\mathbf{x}_i^T\mathbf{x}_j$. After feature mapping, this becomes $\boldsymbol{\phi}(\mathbf{x}_i)^T\boldsymbol{\phi}(\mathbf{x}_j)$. **Key insight:** We define a kernel function $K(\mathbf{x}_i, \mathbf{x}_j) := \boldsymbol{\phi}(\mathbf{x}_i)^T\boldsymbol{\phi}(\mathbf{x}_j)$ that computes this inner product directly in the original space without ever computing $\boldsymbol{\phi}$. **Validity condition (Mercer's theorem):** K must be symmetric and positive semi-definite — i.e., for any set of points, the Gram matrix $[K(\mathbf{x}_i, \mathbf{x}_j)]$ must be PSD.

Standard Kernel Functions

Kernel	Formula	Properties
Linear	$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T\mathbf{x}_j$	Equivalent to no mapping; $\boldsymbol{\phi}(\mathbf{x}) = \mathbf{x}$
Polynomial	$K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma\mathbf{x}_i^T\mathbf{x}_j + \beta)^p$	Captures interactions up to degree p
RBF (Gaussian)	$K(\mathbf{x}_i, \mathbf{x}_j) = e^{-\gamma\lVert\mathbf{x}_i - \mathbf{x}_j\rVert_2^2}$	Infinite-dimensional feature space; local similarity
Hyperbolic Tangent	$K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\gamma\mathbf{x}_i^T\mathbf{x}_j + \beta)$	Neural network-like; not always PSD

Kernelized inference:

$$f(\mathbf{x}) = \sum_{j=1}^{m} \hat{\alpha}_j y_j K(\mathbf{x}_j, \mathbf{x}) + \hat{b}$$

Only support vectors (with $\hat{\alpha}_j > 0$) contribute — the kernel is evaluated only against those points.

Slides 42–44: Choosing SVM Hyperparameters

Parameter C (all kernels)

$$\text{Low } C \to \text{Wide margin, many violations} \to \text{High bias, low variance (underfitting)}$$

$$\text{High } C \to \text{Narrow margin, few violations} \to \text{Low bias, high variance (overfitting)}$$

Parameter γ (RBF kernel) The RBF kernel $K(\mathbf{x}_i, \mathbf{x}_j) = e^{-\gamma\lVert\mathbf{x}_i - \mathbf{x}_j\rVert_2^2}$ has an important geometric interpretation: γ controls the rate of decay of the kernel value with distance. Equivalently, $1/\gamma$ is proportional to the bandwidth (influence radius) of each support vector.

$$\text{Low } \gamma \to \text{Broad influence radius} \to \text{Smooth boundary} \to \text{Underfitting}$$

$$\text{High } \gamma \to \text{Narrow influence radius} \to \text{Spiky boundary} \to \text{Overfitting}$$

Standard practice: Both C and γ are chosen via cross-validation over a grid (e.g., $C \in \{10^{-3}, \ldots, 10^3\}$, $\gamma \in \{10^{-3}, \ldots, 10^3\}$).

3 Conceptual Deepening

Connection to Regularized Empirical Risk Minimization: The soft margin SVM can be written equivalently as an unconstrained problem using the hinge loss $\ell(t) = \max(0, 1-t)$:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{j=1}^{m} \max(0, 1 - y_j(\mathbf{w}^T\mathbf{x}_j + b))$$

This is precisely L2-regularized ERM with hinge loss. The slides do not explicitly state this, but it is a standard equivalence in the literature.

Structural Risk Minimization: The margin $2/\|\mathbf{w}\|_2$ is related to the VC dimension of the hypothesis class. Maximizing the margin corresponds to minimizing an upper bound on the generalization error — this is the theoretical foundation of SVM.

4 Mathematical Clarification

Why does $\inf_{\mathbf{w}} L = -\frac{1}{2}\|\mathbf{w}\|_2^2 + \sum_j \alpha_j$**?** Starting from $\mathbf{w} = \sum_j \alpha_j y_j \mathbf{x}_j$:

$$\frac{1}{2}\|\mathbf{w}\|_2^2 = \frac{1}{2}\mathbf{w}^T\mathbf{w} = \frac{1}{2}\sum_i\sum_j \alpha_i\alpha_j y_i y_j \mathbf{x}_i^T\mathbf{x}_j$$

The cross-term from $\sum_j \alpha_j y_j(\mathbf{x}_j^T\mathbf{w}) = \mathbf{w}^T\mathbf{w} = \|\mathbf{w}\|_2^2$. So: $\frac{1}{2}\|\mathbf{w}\|_2^2 - \|\mathbf{w}\|_2^2 = -\frac{1}{2}\|\mathbf{w}\|_2^2$. Combined with $\sum_j \alpha_j$ (the constant terms), we get the dual objective.

5 Common Pitfalls

- **Pitfall 1 — Confusing w with the decision boundary:** $\mathbf{w}$ is the normal vector to the hyperplane, not the hyperplane itself. The decision boundary is the set $\{\mathbf{x} : \mathbf{w}^T\mathbf{x} + b = 0\}$.
- **Pitfall 2 — Thinking all training points are support vectors:** Only the minority of points on the margin boundary or violating it have $\alpha_j > 0$. The majority have $\alpha_j = 0$ and do not affect the solution.
- **Pitfall 3 — Misinterpreting C:** Large C does not always mean better performance. It means more aggressive fitting to training data, which can overfit. The optimal C is data-dependent.
- **Pitfall 4 — Assuming the kernel always maps to finite dimensions:** The RBF kernel corresponds to an infinite-dimensional feature space. This is why the kernel trick is so powerful — it works without ever specifying or computing $\boldsymbol{\phi}(\mathbf{x})$.
- **Pitfall 5 — Using the primal solution for kernel SVM:** Once a nonlinear kernel is used, $\mathbf{w}$ no longer exists as an explicit finite vector in the original space. Inference must be done through the kernelized dual formula: $f(\mathbf{x}) = \sum_j \hat{\alpha}_j y_j K(\mathbf{x}_j, \mathbf{x}) + \hat{b}$.

6 Practical Interpretation

When to use SVM:

- When the class boundary is determined by the interaction between classes (inter-class geometry matters)
- When $n \gg m$ (high-dimensional, few samples) — primal SVM is efficient
- When $m \gg n$ (many samples, low-dimensional) — dual SVM is efficient

- When you need a sparse model at inference (only support vectors matter)

When to prefer alternatives:

- Very large datasets: SVM training scales as $O(m^2)$ to $O(m^3)$ depending on the solver — neural networks or tree methods may scale better
- Multi-class problems natively: SVM requires one-vs-one or one-vs-all decomposition
- When probabilistic outputs are needed: SVM gives scores, not probabilities (Platt scaling can be applied but is not native)

Compared to logistic regression: Both are linear classifiers with regularization. SVM uses hinge loss + margin maximization; logistic regression uses log-loss + L2 regularization. SVM produces a sparse solution (only support vectors matter); logistic regression uses all training points.

7 Summary

Concept	**Key Formula/Idea**
Decision function	$f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$
Hard margin objective	$\min \frac{1}{2}\|\mathbf{w}\|_2^2$ s.t. $y_j(\mathbf{w}^T\mathbf{x}_j + b) \geq 1$
Soft margin objective	$\min \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_j z_j$ s.t. $y_j(\mathbf{w}^T\mathbf{x}_j + b) \geq 1 - z_j$
Margin width	$2/\|\mathbf{w}\|_2$
Dual variable recovery	$\hat{\mathbf{w}} = \sum_j \hat{\alpha}_j y_j \mathbf{x}_j$
Kernel substitution	Replace $\mathbf{x}_i^T\mathbf{x}_j$ with $K(\mathbf{x}_i, \mathbf{x}_j)$ in dual
Support vectors	Points with $\hat{\alpha}_j > 0$; define the solution
C effect	Large C $\rightarrow$ narrow margin $\rightarrow$ overfitting
γ effect (RBF)	Large $\gamma \rightarrow$ narrow influence $\rightarrow$ overfitting

Three conceptual pillars of SVM:

1. Margin maximization — the geometric principle that drives generalization
2. Convex optimization — guarantees a unique, globally optimal solution
3. The kernel trick — extends linear SVM to nonlinear boundaries without explicitly computing feature maps